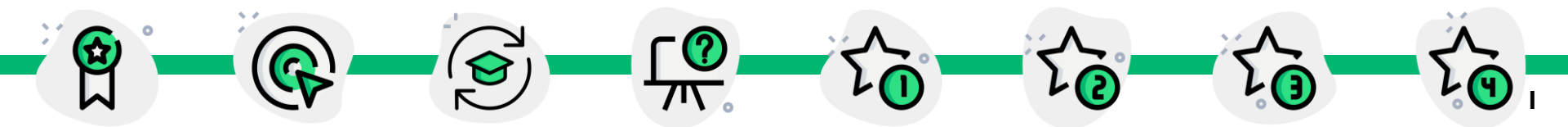


「2026년도 1학기 한국성서대학교 AI융합학부」

알고리즘

2026.03.30

4주차



스택과 큐

04-1 스택이란?

04-2 큐란?



04-1 스택이란?

스택 알아보기

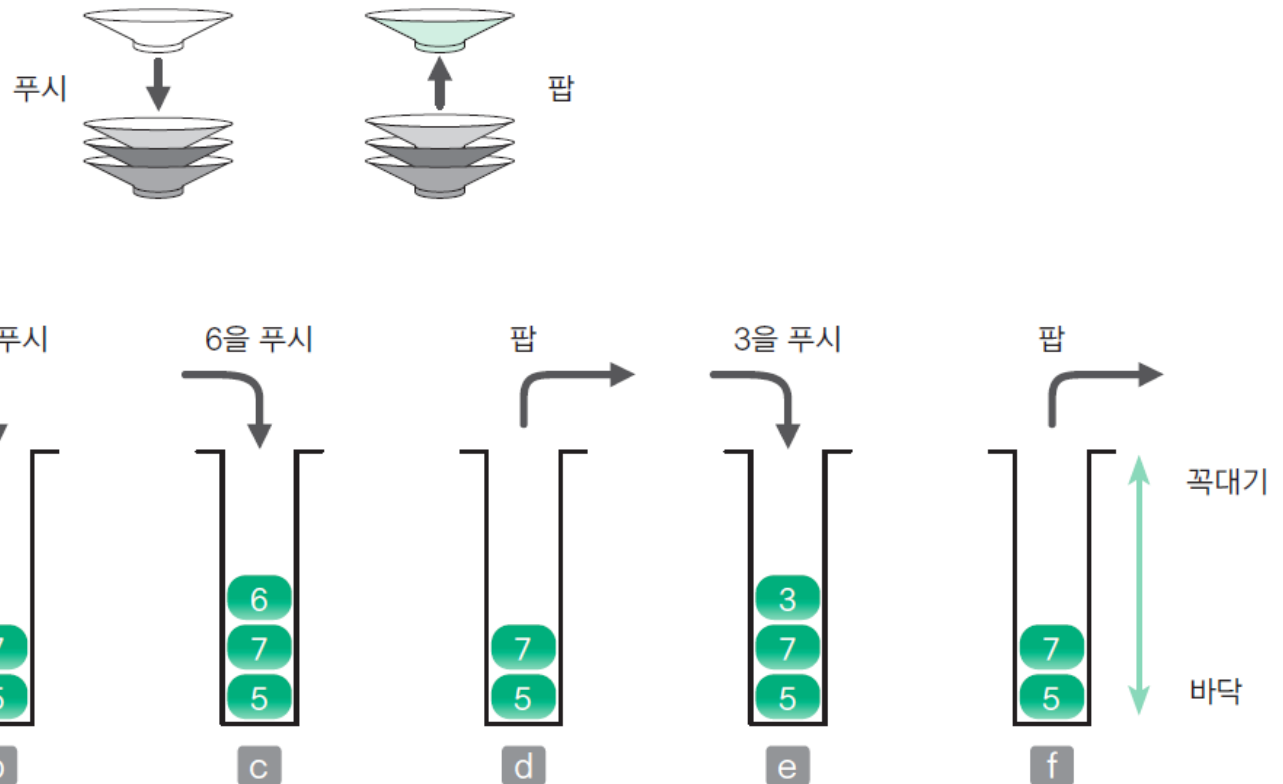
◆ 스택stack

- 데이터를 일시적으로 저장하기 위해 사용하는 자료구조
- 데이터의 입력과 출력 순서는 후입선출(LIFO, Last In First Out)
- 즉, 가장 나중에 넣은 데이터를 가장 먼저 꺼냄
- 푸시(push): 스택에 데이터를 넣는 작업
- 팝(pop): 스택에서 데이터를 꺼내는 작업
- 꼭대기(top): 푸시, 팝을 하는 위치
 - ✓ 테이블에 겹겹이 쌓은 접시처럼 데이터를 넣는 작업도 꺼내는 작업도 위쪽부터 수행
- 바닥(bottom): 스택의 가장 밑바닥 부분



04-1 스택이란?

스택 알아보기



[그림 4-1] 스택의 푸시와 팝



04-1 스택이란?

◆ 스택의 구현의 예(1/2)

- 함수를 호출하고 실행할 때 프로그램 내부에서는 스택을 사용
- 가장 먼저 main 함수를 실행하며 main 함수는 z 함수를 호출
- 호출된 함수 z는 x 함수와 y 함수를 순서대로 호출
- 함수를 호출할 때는 푸시하고 함수가 실행을 종료하고 호출한 원래의 함수로 돌아갈 때는 종료할 함수를 팝함
- **d** main → z → x의 순서대로 함수를 호출
 - ✓ 이때 스택의 상태는 호출한 함수의 역순으로 겹겹이 쌓여 있어 함수 호출이 계층 구조로 이루어져 있음
 - ✓ 이 상태에서 x 함수의 실행이 종료되면 x 함수만 팝함

다음 페이지 그림 확인!

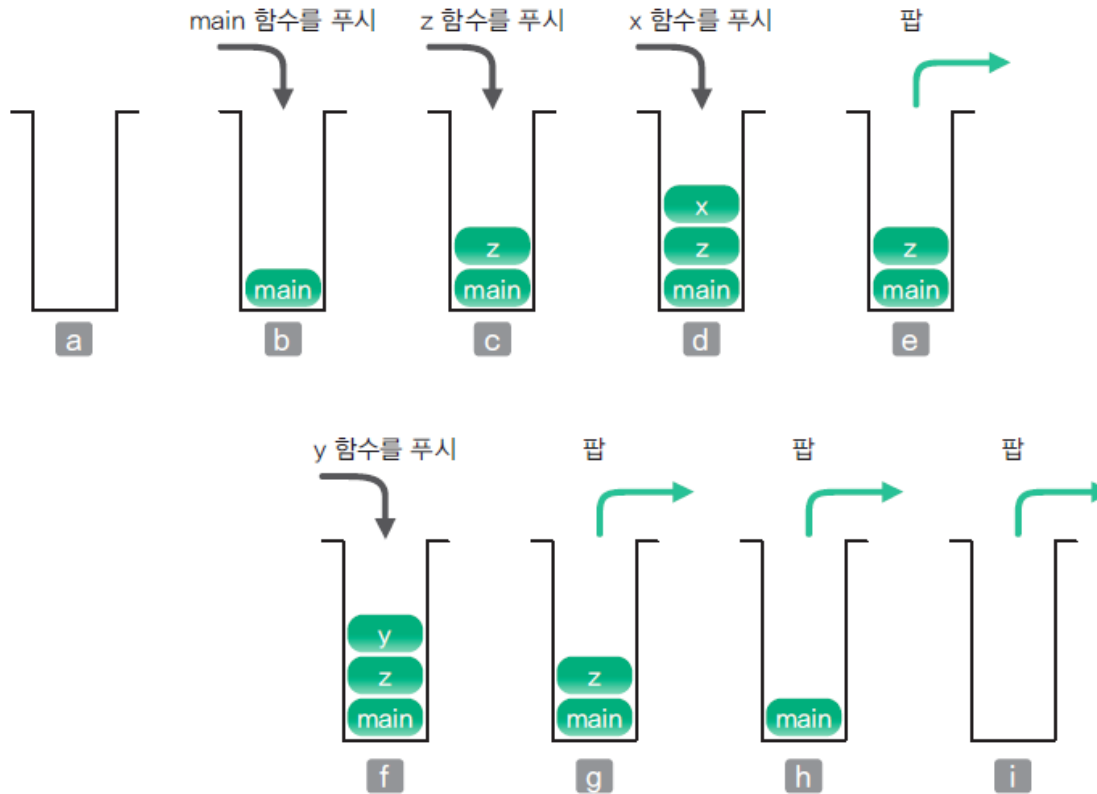


04-1 스택이란?

◆ 스택의 구현의 예(1/2)

```
void x() { /*...*/ }  
void y() { /*...*/ }  
void z() {  
    x();  
    y();  
}  
int main(void) {  
    z();  
}
```

- a main 함수가 실행되기 전의 상태입니다.
- b main 함수를 실행합니다.
- c z 함수를 호출합니다.
- d x 함수를 호출합니다.
- e x 함수가 종료되고 z 함수로 돌아옵니다.
- f y 함수를 호출합니다.
- g y 함수를 종료하고 z 함수로 돌아옵니다.
- h z 함수를 종료하고 main 함수로 돌아옵니다.
- i main 함수를 종료합니다.



[그림 4-2] 함수 호출과 스택



04-1 스택이란?

◆ 스택 구조체

- 스택 구조체 IntStack
 - 스택으로 사용할 배열을 가리키는 포인터 stk
 - 인덱스가 0인 요소를 스택의 바닥(bottom)이라 함
 - 배열의 메모리 공간 할당은 Initialize 함수로 생성
 - 스택의 최대 용량 max
 - 배열 stk의 요소 개수와 같음
 - 스택 포인터 ptr
 - 스택에 쌓여 있는 데이터의 개수를 나타냄
 - 스택이 비어 있으면 ptr의 값은 0이고 가득 차 있으면 max



04-1 스택이란?

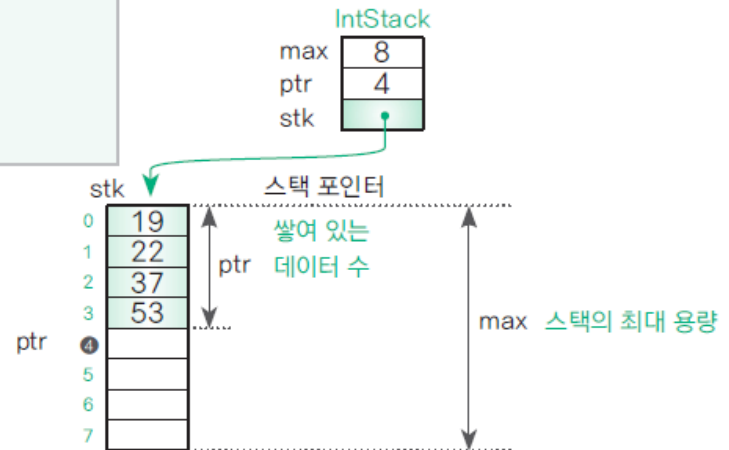
◆ 스택 구조체

Do it! 실습 4-1

• 완성 파일 chap04/IntStack.h

```
01 // int형 스택 IntStack(헤더)
02 #ifndef __IntStack
03 #define __IntStack
04
05 /*--- 스택을 구현하는 구조체 ---*/
06 typedef struct {
07     int max;    // 스택 용량
08     int ptr;    // 스택 포인터
09     int *stk;   // 스택의 첫 요소에 대한 포인터
10 } IntStack;
```

```
typedef struct {
    int max;    // 스택 용량
    int ptr;    // 스택 포인터
    int *stk;   // 스택을 가리키는 포인터
} IntStack;
```



[그림 4-3] 스택의 구현 예



04-1 스택이란?

◆ 스택 만들기

```
12  /*--- 스택 초기화 ---*/
13  int Initialize(IntStack *s, int max);
14
15  /*--- 스택에 데이터를 푸시---*/
16  int Push(IntStack *s, int x);
17
18  /*--- 스택에서 데이터를 팝 ---*/
19  int Pop(IntStack *s, int *x);
20
21  /*--- 스택에서 데이터를 피크 ---*/
22  int Peek(const IntStack *s, int *x);
23
24  /*--- 스택 비우기 ---*/
25  void Clear(IntStack *s);
26
27  /*--- 스택의 최대 용량 ---*/
28  int Capacity(const IntStack *s);
29
```

```
30  /*--- 스택의 데이터 개수 ---*/
31  int Size(const IntStack *s);
32
33  /*--- 스택이 비어 있나요? ---*/
34  int IsEmpty(const IntStack *s);
35
36  /*--- 스택이 가득 찼나요? ---*/
37  int IsFull(const IntStack *s);
38
39  /*--- 스택에서 검색 ---*/
40  int Search(const IntStack *s, int x);
41
42  /*--- 모든 데이터 출력 ---*/
43  void Print(const IntStack *s);
44
45  /*--- 스택 종료 ---*/
46  void Terminate(IntStack *s);
47  #endif
```



04-1 스택이란?

◆ 스택 초기화 함수 Initialize

- 스택의 메모리 공간(배열)을 확보하는 등의 준비 작업을 수행하는 함수
- **1** 배열을 위한 메모리 공간을 만들 때 스택은 비어 있어야(데이터가 하나도 쌓여 있지 않은 상태여야) 하므로 스택 포인터 ptr 값을 0으로 함
- **2** 요소의 개수가 max인 배열 stk를 생성함. 이때, 스택의 개별 요소에 접근하는 인덱스 식은 바닥(bottom)부터 $stk[0]$, $stk[1]$, ..., $stk[\text{max} - 1]$ 이 됨
- **3** 매개변수 max로 받은 값을 스택 최대 용량을 나타내는 구조체의 멤버 max에 저장



04-1 스택이란?

◆ 스택 초기화 함수 Initialize

Do it! 실습 4-2[A]

• 완성 파일 chap04/IntStack.c

```
01 // int형 스택 IntStack(소스)
02 #include <stdio.h>
03 #include <stdlib.h>
04 #include "IntStack.h"
05
06 /*--- 스택 초기화 ---*/
07 int Initialize(IntStack *s, int max)
08 {
09     s->ptr = 0;
10     if((s->stk = calloc(max, sizeof(int))) == NULL) {
11         s->max = 0; // 배열의 생성에 실패
12         return -1;
13     }
14     s->max = max;
15     return 0;
16 }
17
```

- calloc(max, sizeof(int))은 max개의 정수를 저장할 수 있는 배열을 동적으로 생성하고, 모든 요소를 0으로 초기화한 뒤 배열의 첫 번째 요소 주소를 반환
- s->stk = calloc(...)로 할당된 주소를 s->stk에 저장한 뒤, 이 값이 NULL인지 확인합니다. 만약 NULL이라면, 메모리 할당에 실패했음을 의미함

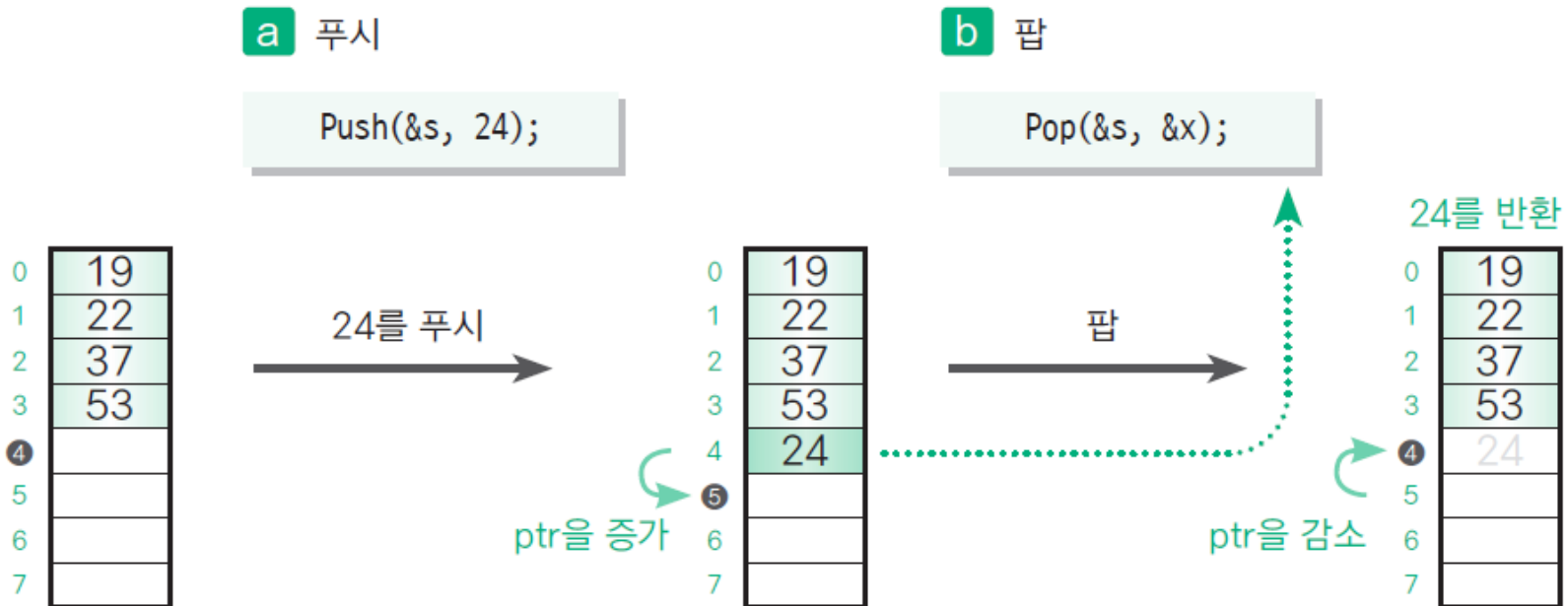
(실습 4-2[B]에서 계속)



04-1 스택이란?

◆ 푸시 함수 Push

- 스택에 데이터를 추가하는 함수
- 새로 추가할 데이터(x)를 배열의 요소 `stk[ptr]`에 저장하고 스택 포인터 `ptr`을 증가



[그림 4-4] 스택에서의 푸시와 팝



04-1 스택이란?

◆ 푸시 함수 Push

Do it! 실습 4-2[B]

• 완성 파일 chap04/IntStack.c

```
01  /*--- 스택에 데이터 푸시 ---*/
02  int Push(IntStack *s, int x)
03  {
04      if (s->ptr >= s->max)
05          return -1;
06      s->stk[s->ptr++] = x;
07      return 0;
08  }
09
```

// 스택이 가득 참

→ 후위 증가 연산자 : 현재 ptr이 가리키는 위치에 x 저장
그 뒤 ptr++ → ptr을 1 증가 (다음 위치를 가리키도록 변경)

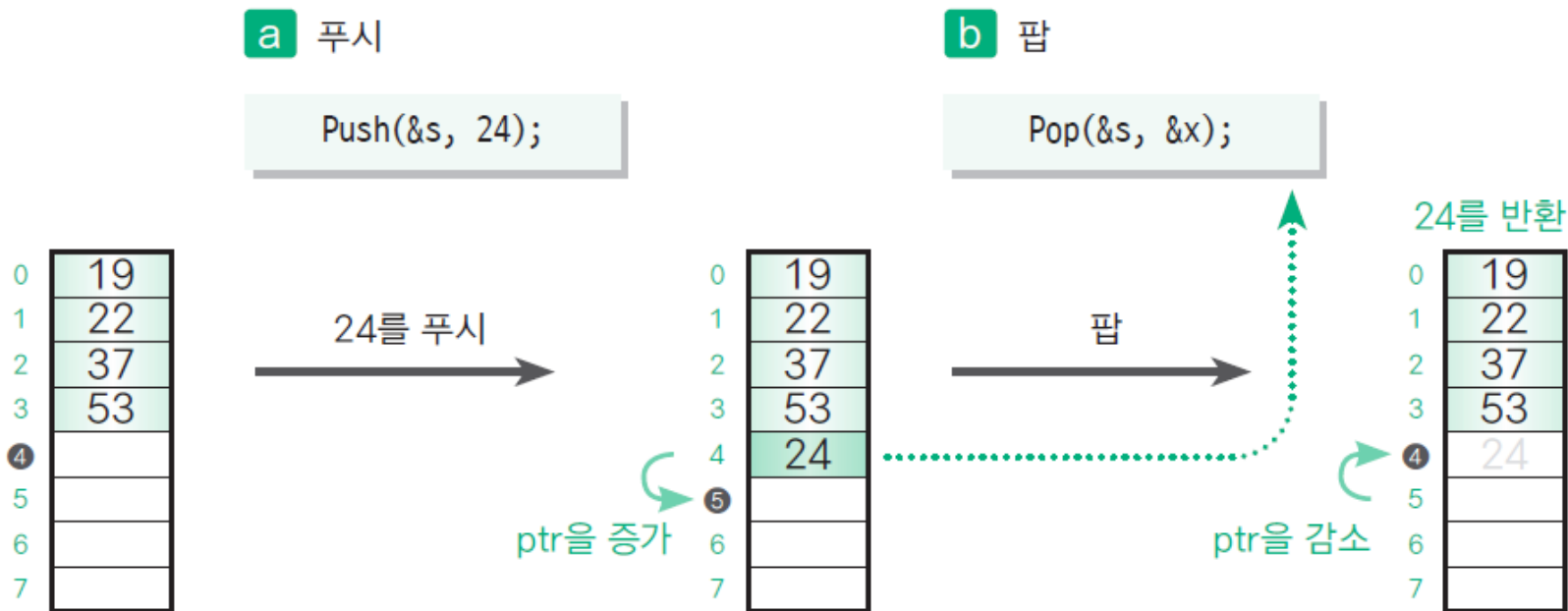
(실습 4-2[C]에서 계속)



04-1 스택이란?

◆ 팝 함수 Pop

- 스택의 꼭대기에서 데이터를 제거하는 함수
- 스택 포인터 ptr의 값을 감소시키고 stk[ptr]에 저장된 값을 포인터 x가 가리키는 변수에 저장



[그림 4-4] 스택에서의 푸시와 팝



04-1 스택이란?

◆ 팝 함수 Pop

Do it! 실습 4-2[C]

• 완성 파일 chap04/IntStack.c

```
01  /*--- 스택에서 데이터를 제거---*/
02  int Pop(IntStack *s, int *x)
03  {
04      if (s->ptr <= 0)    // 스택이 비어 있음
05          return -1;
06      *x = s->stk[--s->ptr];
07      return 0;
08  }
09
```

(실습 4-2[D]에서 계속)



04-1 스택이란?

◆ 피크 함수 Peek

- 스택 꼭대기의 데이터를 '몰래 엿보는' 함수
- 스택이 비어 있지 않으면 꼭대기 요소 `stk[ptr - 1]`의 값을 포인터 `x`가 가리키는 변수에 저장
- 데이터의 입력과 출력이 없으므로 스택 포인터는 변화하지 않음

Do it! 실습 4-2[D]

• 완성 파일 chap04/IntStack.c

```
01  /*--- 스택에서 데이터 엿보기 ---*/
02  int Peek(const IntStack *s, int *x)
03  {
04      if (s->ptr <= 0)          // 스택이 비어 있음
05          return -1;
06      *x = s->stk[s->ptr - 1];
07      return 0;
08  }
09
```

(실습 4-2[E]에서 계속)



04-1 스택이란?

◆ 스택의 모든 요소를 삭제하는 함수 Clear

- 스택에 대한 푸시와 팝 등 모든 작업은 스택 포인터를 바탕으로 이루어짐
- 스택의 배열 요솟값을 변경할 필요가 없고 모든 요소의 삭제는 스택 포인터 ptr값을 0으로 하면 끝남

Do it! 실습 4-2[E]

• 완성 파일 chap04/IntStack.c

```
01  /*--- 스택의 모든 요소 삭제 ---*/  
02  void Clear(IntStack *s)  
03  {  
04      s->ptr = 0;  
05  }  
06
```

(실습 4-2[F]에서 계속)



04-1 스택이란?

◆ 용량을 확인하는 함수 Capacity

- 스택의 용량(멤버 max의 값)을 반환하는 함수

◆ 데이터의 개수를 확인하는 함수 Size

- 현재 스택에 쌓여 있는 데이터의 개수(멤버 ptr의 값)를 반환하는 함수

◆ 스택이 비어 있는지 검사하는 함수 IsEmpty

- 스택이 비어 있으면 1, 그렇지 않으면 0을 반환

◆ 스택이 가득 찼는지 검사하는 함수 IsFull

- 스택이 가득 찼으면 1, 그렇지 않으면 0을 반환



04-1 스택이란?

Do it! 실습 4-2[F]

• 완성 파일 chap04/IntStack.c

```
01  /*--- 스택 용량 ---*/
02  int Capacity(const IntStack *s)
03  {
04      return s->max;
05  }
06
07  /*--- 스택에 쌓여 있는 데이터 수 ---*/
08  int Size(const IntStack *s)
09  {
```



04-1 스택이란?

Do it! 실습 4-2[F]

• 완성 파일 chap04/IntStack.c

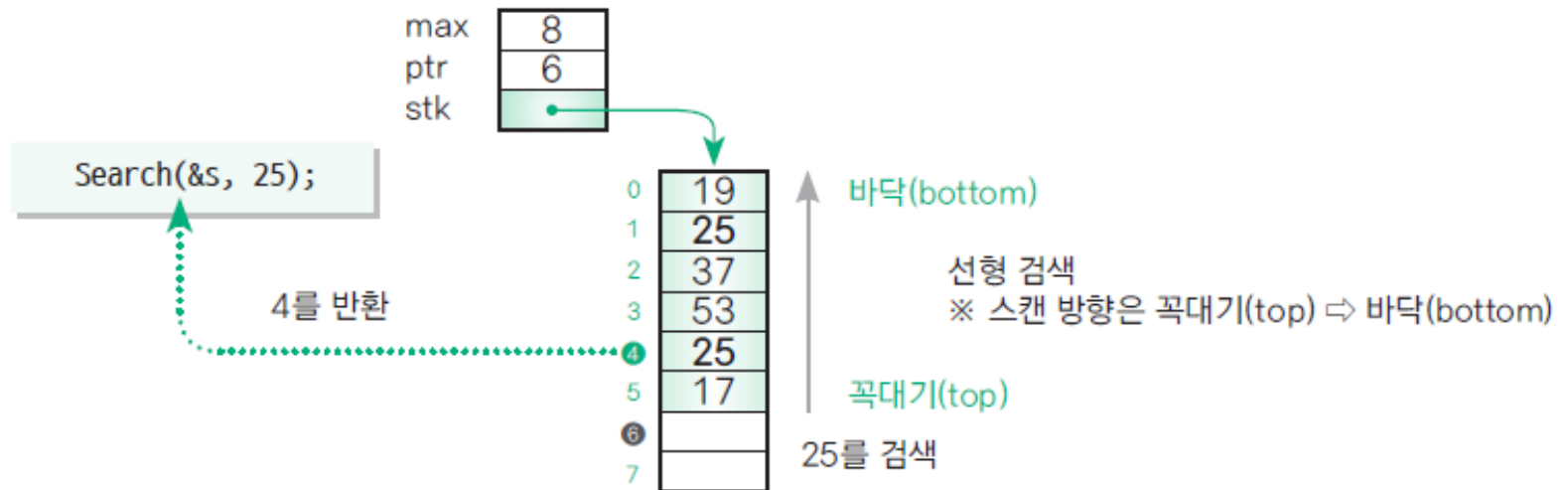
```
10     return s->ptr;
11 }
12
13 /*--- 스택이 비어 있는가? ---*/
14 int IsEmpty(const IntStack *s)
15 {
16     return s->ptr <= 0;
17 }
18
19 /*--- 스택은 가득 찼는가? ---*/
20 int IsFull(const IntStack *s)
21 {
22     return s->ptr >= s->max;
23 }
```



04-1 스택이란?

◆ 임의의 값을 검색하는 함수 Search

- 임의의 값 x의 데이터가 스택의 어느 위치에 쌓여 있는지 검사하는 함수
- 검색은 꼭대기에서 바닥으로 선형 검색을 수행
- 꼭대기부터 검색하는 이유는 ‘먼저 팝되는 데이터’를 찾기 위함
- 검색에 성공하면 찾은 요소의 인덱스를 반환하고 실패하면 -1을 반환



[그림 4-5] 스택에서 검색



04-1 스택이란?

```
25  /*--- 스택에서 검색 ---*/
26  int Search(const IntStack *s, int x)
27  {
28      for(int i = s->ptr - 1; i >= 0; i--)    // 꼭대기(top) → 바닥(bottom)으로 선형 검색
29          if(s->stk[i] == x)
30              return i;                      // 검색 성공
31      return -1;                             // 검색 실패
32  }
```



04-1 스택이란?

◆ 모든 데이터를 출력하는 함수 Print

- 스택의 모든 데이터를 바닥부터 순서대로 출력하는 함수

◆ 종료 함수 Terminate

- Initialize 함수로 확보한 스택을 해제하고 용량 max와 스택 포인터 ptr의 값을 0으로 함



04-1 스택이란?

```
34  /*--- 모든 데이터 출력 ---*/
35  void Print(const IntStack *s)
36  {
37      int i;
38      for(int i = 0; i < s->ptr; i++)          // 바닥(bottom) → 꼭대기(top)로 스캔
39          printf("%d ", s->stk[i]);
40      putchar( '\n');
41  }
42
43  /*--- 스택 종료 ---*/
44  void Terminate(IntStack *s)
45  {
46      if(s->stk != NULL)
47          free(s->stk);                        // 배열을 삭제
48      s->max = s->ptr = 0;
49  }
```




04-1 스택이란?

◆ 스택만들기 예제

▪ 예제 4-3

Chapt04_예제 4-3/IntStackTest.c

// int형 스택 IntStack의 사용

```
#include <stdio.h>
```

```
#include "IntStack.h"
```

```
int main(void)
```

```
{
```

```
    IntStack s;
```

```
    if (Initialize(&s, 64) == -1) {
```

```
        puts("스택 생성에 실패했습니다.");
```

```
        return 1;
```

```
    }
```



04-1 스택이란?

```
while (1) {
    int menu, x;
    printf("현재 데이터 수: %d / %d\n", Size(&s), Capacity(&s));
    printf("(1)푸시 (2)팝 (3)피크 (4)출력 (0)종료: ");
    scanf("%d", &menu);

    if (menu == 0) break;
    switch (menu) {
        case 1: /*--- 푸시 ---*/
            printf("데이터: ");
            scanf("%d", &x);
            if (Push(&s, x) == -1) //if문 뒤에는 푸시 성공에 대한 else가 생략되어짐
                puts("\a오류: 푸시에 실패했습니다.");
            break;

        case 2: /*--- 팝 ---*/
            if (Pop(&s, &x) == -1)
                puts("\a오류: 팝에 실패했습니다.");
            else
                printf("팝 데이터는 %d입니다.\n", x);
            break;
```



04-1 스택이란?

case 3: /*--- 피크 ---*/

```
    if (Peek(&s, &x) == -1)
```

```
        puts("\a오류: 피크에 실패했습니다.");
```

```
    else
```

```
        printf("피크 데이터는 %d입니다.\n", x);
```

```
    break;
```

case 4: /*--- 출력 ---*/

```
    Print(&s);
```

```
    break;
```

```
}
```

```
}
```

```
Terminate(&s);
```

```
return 0;
```

```
}
```



04-1 스택이란?

◆ 스택만들기 예제 - 실행 결과

실행 결과

현재 데이터 수: 0/64

(1) 푸시 (2) 팝 (3) 피크 (4) 출력 (0) 종료: 1

데이터: 1

1을 푸시

현재 데이터 수: 1/64

(1) 푸시 (2) 팝 (3) 피크 (4) 출력 (0) 종료: 1

데이터: 2

2를 푸시

현재 데이터 수: 2/64

(1) 푸시 (2) 팝 (3) 피크 (4) 출력 (0) 종료: 1

데이터: 3

3을 푸시

현재 데이터 수: 3/64

(1) 푸시 (2) 팝 (3) 피크 (4) 출력 (0) 종료: 1

데이터: 4

4를 푸시

현재 데이터 수: 4/64

(1) 푸시 (2) 팝 (3) 피크 (4) 출력 (0) 종료: 3

피크 데이터는 4입니다.

4를 피크

현재 데이터 수: 4/64

(1) 푸시 (2) 팝 (3) 피크 (4) 출력 (0) 종료: 4

1 2 3 4

스택의 내용을 출력

현재 데이터 수: 4/64

(1) 푸시 (2) 팝 (3) 피크 (4) 출력 (0) 종료: 2

팝 데이터는 4입니다.

4를 팝

현재 데이터 수: 3/64

(1) 푸시 (2) 팝 (3) 피크 (4) 출력 (0) 종료: 2

팝 데이터는 3입니다.

3을 팝

현재 데이터 수: 2/64

(1) 푸시 (2) 팝 (3) 피크 (4) 출력 (0) 종료: 4

1 2

스택의 내용을 출력

현재 데이터 수: 2/64

(1) 푸시 (2) 팝 (3) 피크 (4) 출력 (0) 종료: 0



04-2 큐란?

큐 알아보기

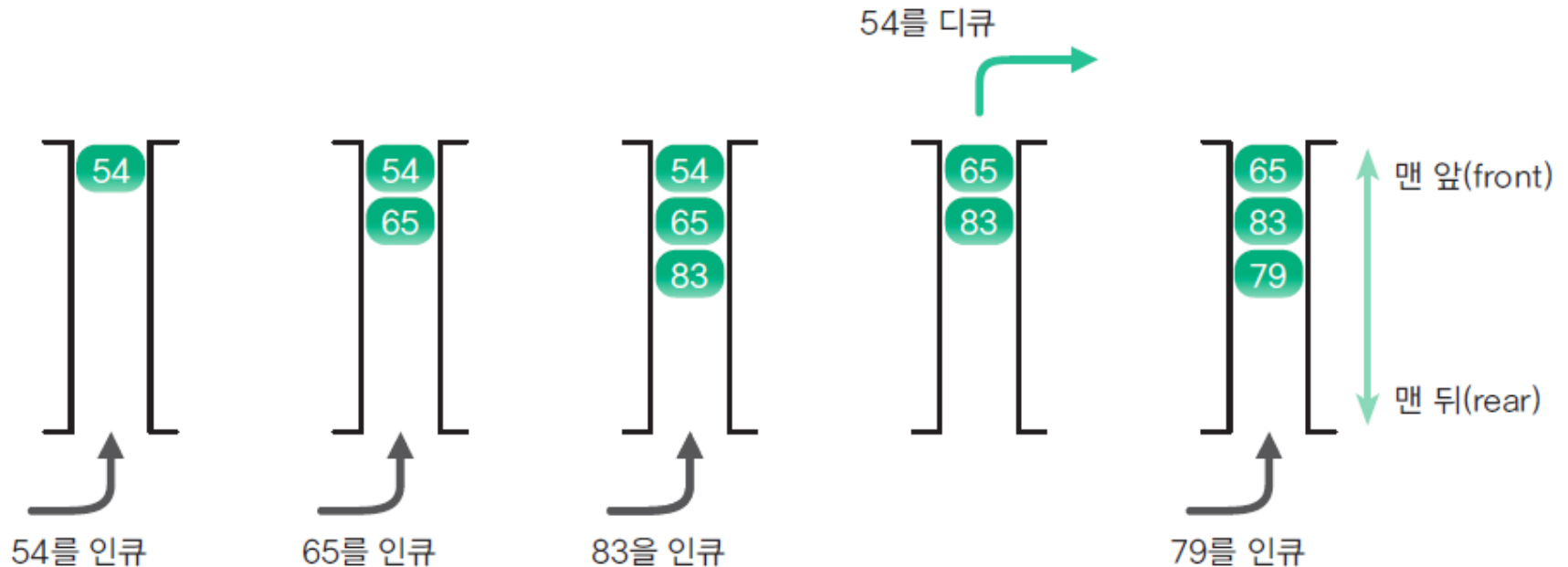
◆ 큐queue

- 데이터를 일시적으로 저장하기 위해 사용하는 자료구조
- 데이터를 일시적으로 쌓아 두기 위한 자료구조
- 가장 먼저 넣은 데이터를 가장 먼저 꺼내는 선입선출 구조를 이룸
- 은행 창구에서 차례를 기다리는 대기열이나 마트에서 계산을 기다리는
- 대기열과 같음
- 인큐(en-queue): 큐에 데이터를 넣는 작업
- 디큐(de-queue): 데이터를 꺼내는 작업
- 프론트(front): 데이터를 꺼내는 쪽
- 리어(rear): 데이터를 넣는 쪽



04-2 큐란?

큐 알아보기

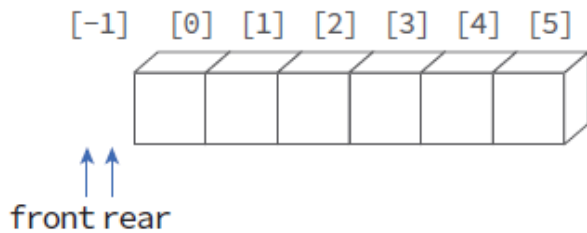


[그림 4-6] 인큐와 디큐

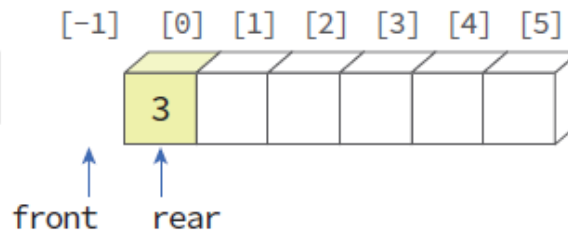


04-2 큐란?

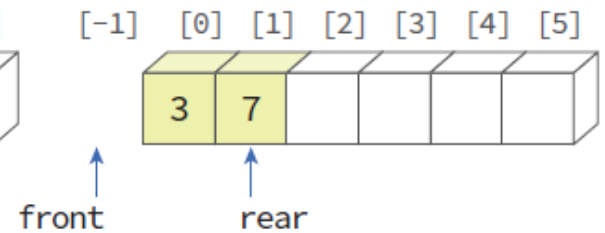
큐 알아보기



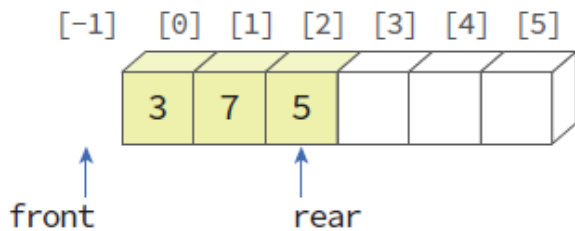
(a) 초기상태



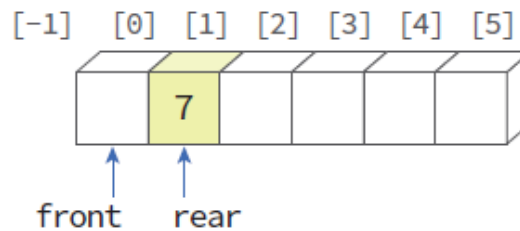
(b) enqueue(3)



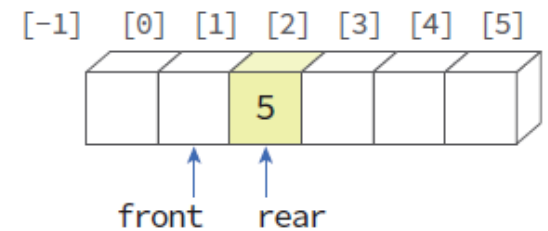
(c) enqueue(7)



(d) enqueue(5)



(e) dequeue()



(f) dequeue()



04-2 큐란?

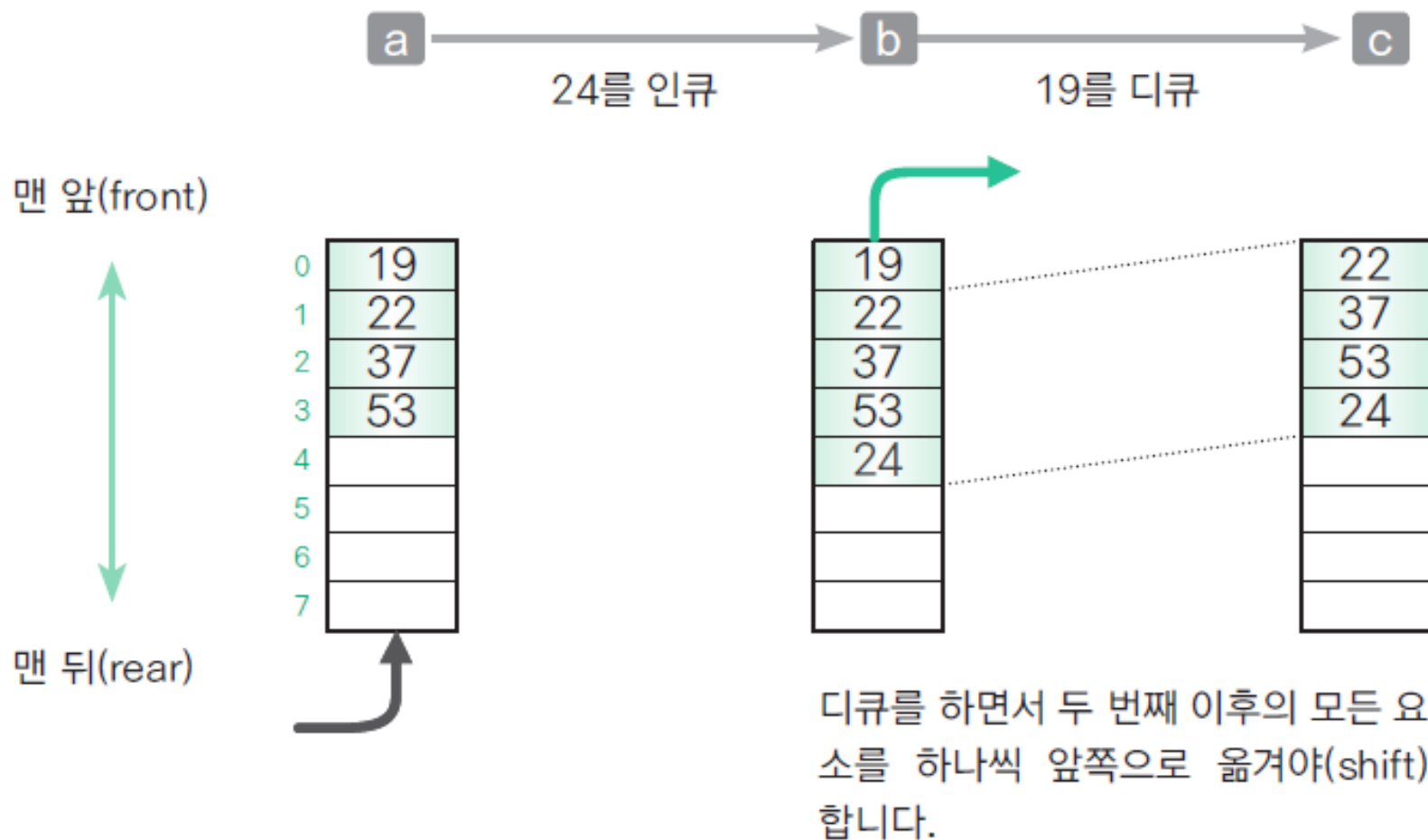
◆ 배열로 큐 만들기

- **a** 배열의 프론트(front)부터 4개(19, 22, 37, 53)의 데이터가 들어가 있는 모습
- **b** 24 인큐
 - ✓ 리어(rear)의 데이터가 저장된 `que[3]`의 다음 요소 `que[4]`에 24를 저장
 - ✓ 이 처리의 복잡도는 $O(1)$ 이고 적은 비용으로 구현할 수 있음
- **c** 19 디큐
 - ✓ `que[0]`에 저장된 19를 꺼낸 다음 두 번째 이후의 요소를 모두 맨 앞으로 옮김
 - ✓ 이 처리의 복잡도는 $O(n)$ 이며 데이터를 꺼낼 때마다 이런 처리를 하게 되면 효율이 떨어짐

다음 페이지 그림 참고!



04-2 큐란?



[그림 4-7] 배열에 의한 큐의 구현 예



04-2 큐란?

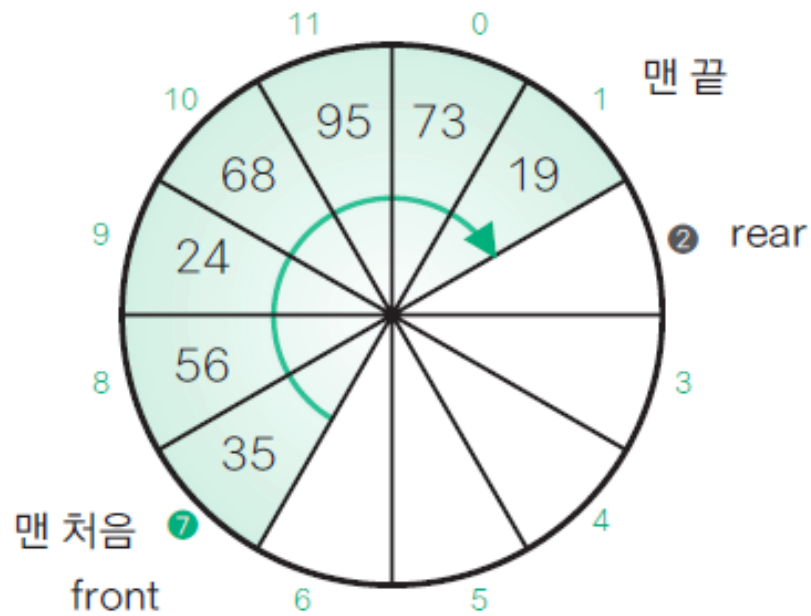
◆ 링버퍼로 큐 만들기(원형큐)

- 배열의 프론트(front)부터 4개(19, 22, 37, 53)의 데이터가 들어가 있는 모습
- 배열의 처음이 끝과 연결되었다고 보는 자료구조
- 프론트(front): 논리적인 맨 처음 요소의 인덱스
- 리어(rear): 논리적인 맨 끝 요소의 하나 뒤의 인덱스(다음 요소를 인큐할 위치를 미리 지정
- 변수 프론트와 리어의 값은 인큐와 디큐를 수행함에 따라 변화하는데 이 때문에 배열 요소를 앞쪽으로 옮기지 않는 큐를 구현할 수 있음
- 처리의 복잡도는 $O(1)$

다음 페이지 그림 참고!



04-2 큐란?



0	73
1	19
2	
3	
4	
5	
6	
7	35
8	56
9	24
10	68
11	95



[그림 4-8] 링 버퍼를 사용하는 큐의 구현



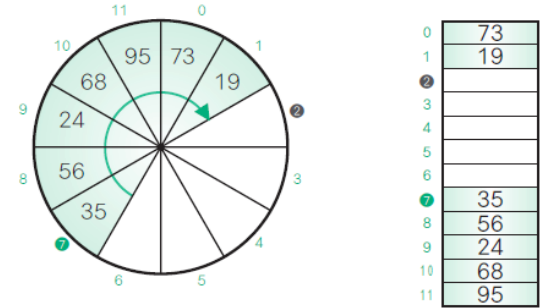
04-2 큐란?

◆ 링버퍼로 큐 만들기(원형큐)

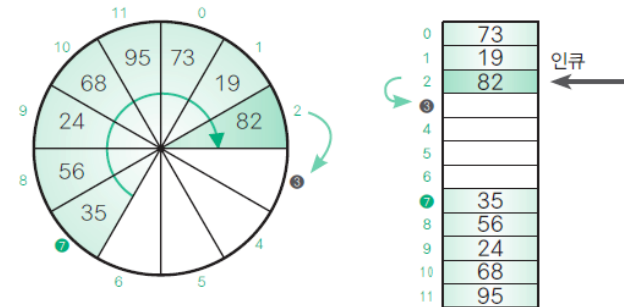
■ 링버퍼에 대한 인큐와 디큐

- a 7개의 데이터(35, 56, 24, 68, 95, 73, 19)가 차례대로 $que[7]$ $que[8]$, ..., $que[11]$, $que[0]$, $que[1]$ 에 저장
 - 프런트값은 7이고 리어값은 2
- b 82를 인큐한 다음의 상태
 - $que[2]$ (리어가 가리키고 있는 위치)에 82를 저장한 다음 리어값을 1만큼 증가
- c 35를 디큐한 다음의 상태
 - 프런트 요소($que[front]$, 즉 $que[7]$)의 값 35를 빼고 프런트값을 1만큼 증가

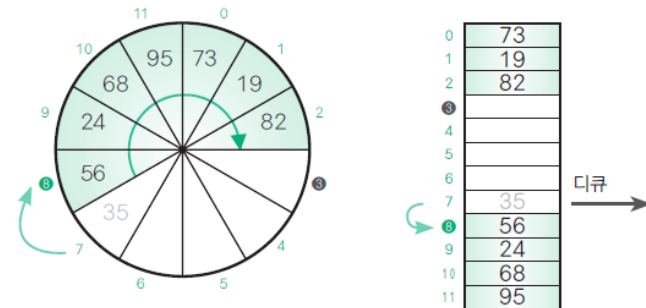
a 큐의 어떤 상태



b 82를 인큐



c 35를 디큐



[그림 4-9] 링 버퍼에 대한 인큐와 디큐



04-2 큐란?

◆ 큐 구조체 IntQueue

- 1. 큐로 사용할 배열(que)
 - ✓ 큐로 사용할 배열의 첫 번째 요소에 대한 포인터
- 2. 큐의 최대 용량(max)
 - ✓ 배열 que에 저장할 수 있는 최대 요소의 개수와 같음
- 3. 프론트(front)
 - ✓ 첫 번째 요소의 인덱스를 저장하는 멤버
- 4. 리어(rear)
 - ✓ 맨 나중에 넣은 요소의 하나 뒤의 인덱스를 저장하는 멤버

다음 페이지 코드 참고!



04-2 큐란?

◆ 큐 구조체 IntQueue

Do it! 실습 4-4

• 완성 파일 chap04/IntQueue.h

```
01 // int형 큐 IntQueue(헤더)
02 #ifndef __IntQueue
03 #define __IntQueue
04
05 /*--- 큐를 구현하는 구조체 ---*/
06 typedef struct {
07     int max;           // 큐의 최대 용량
08     int num;           // 현재의 요소 개수
09     int front;         // 프런트
10     int rear;          // 리어
11     int *que;          // 큐의 맨 앞 요소에 대한 포인터
12 } IntQueue;
```



04-2 큐란?

◆ 큐 구조체 IntQueue

■ 5. 현재 데이터 개수(num)

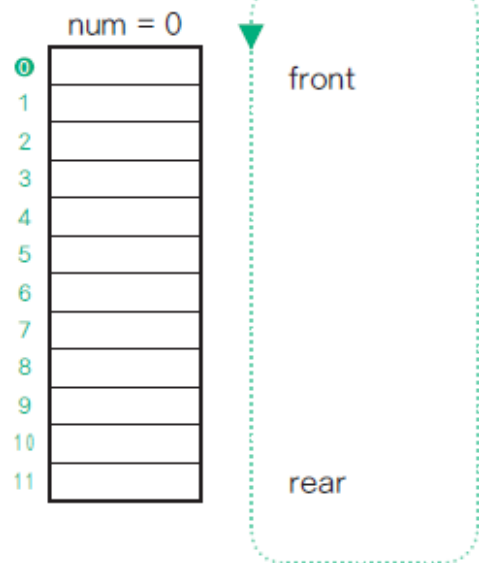
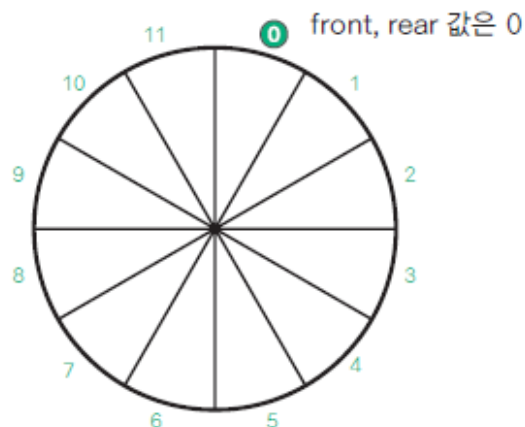
- ✓ 큐에 쌓아 놓은 데이터 수를 나타내는 멤버
- ✓ 큐가 비어 있을 때 num은 0이고, 가득 찼을 때는 num과 max의 값이 같음
- ✓ front와 rear의 값이 같은 경우 큐가 비어 있는지, 가득 찼는지 구별할 수 없는 상황을 피하기 위해 이 변수가 필요
- ✓ 그림 a는 큐가 비어 있는 상태이고 반대로 그림 b는 큐가 가득 찬 상태
- ✓ 두 그림은 전부 front와 rear의 값이 같습니다. 이렇게 num과 max가 없다면 front와 rear 값만으로는 두 상태를 구분할 수 없음

다음 페이지 그림 참고!

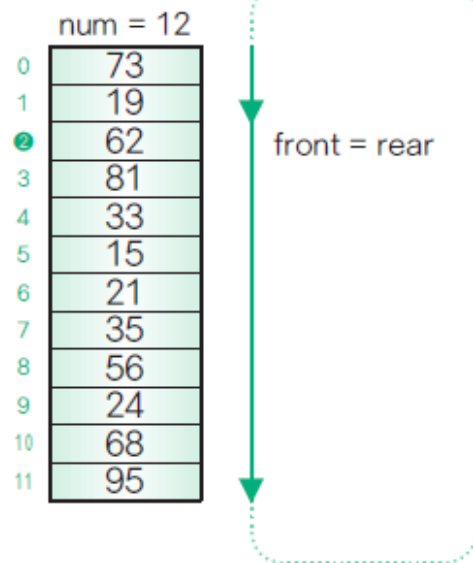
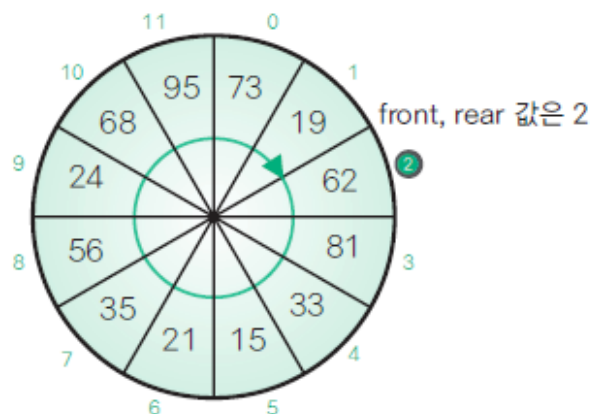


04-2 큐란?

a 큐가 비어 있는 상태(num = 0)



b 큐가 가득 찬 상태(num = 12)



[그림 4-10] 비어 있는 큐와 가득 찬 큐



04-2 큐란?

◆ 큐 만들기

```
14  /*--- 큐 초기화 ---*/
15  int Initialize(IntQueue *q, int max);
16
17  /*--- 큐에 데이터를 인큐 ---*/
18  int Enqueue(IntQueue *q, int x);
19
20  /*--- 큐에서 데이터를 디큐 ---*/
21  int Dequeue(IntQueue *q, int *x);
22
23  /*--- 큐에서 데이터를 피크 ---*/
24  int Peek(const IntQueue *q, int *x);
25
26  /*--- 모든 데이터 삭제 ---*/
27  void Clear(IntQueue *q);
28
29  /*--- 큐의 최대 용량 ---*/
30  int Capacity(const IntQueue *q);
```

```
32  /*--- 큐에 저장된 데이터 개수 ---*/
33  int Size(const IntQueue *q);
34
35  /*--- 큐가 비어 있는가? ---*/
36  int IsEmpty(const IntQueue *q);
37
38  /*--- 큐가 가득 찼는가? ---*/
39  int IsFull(const IntQueue *q);
40
41  /*--- 큐에서 검색 ---*/
42  int Search(const IntQueue *q, int x);
43
44  /*--- 모든 데이터 출력 ---*/
45  void Print(const IntQueue *q);
46
47  /*--- 큐 종료 ---*/
48  void Terminate(IntQueue *q);
49  #endif
```



04-2 큐란?

◆ 초기화 함수 Initialize

- Initialize 함수는 큐를 구현하기 위한 배열 생성 등의 준비 작업을 하는 함수(이때 첫 번째 인수 q는 처리 대상인 큐 구조체 객체에 대한 포인터이며, 이후 대부분의 함수도 마찬가지)
- 큐를 처음 만들면 큐는 비어 있으므로(데이터가 하나도 없는 상태) num, front, rear 값을 모두 0으로 함
- 매개변수 max로 받은 '큐의 최대 용량'을 멤버 max에 저장
- 저장할 수 있는 요소의 개수가 max인 배열 que의 메모리 공간을 확보

다음 페이지 코드 참고!



04-2 큐란?

◆ 초기화 함수 Initialize

Do it! 실습 4-5[A]

```
01 // int형 큐 IntQueue(소스)
02 #include <stdio.h>
03 #include <stdlib.h>
04 #include "IntQueue.h"
05
06 /*--- 큐 초기화 ---*/
```

```
07 int Initialize(IntQueue *q, int max)
08 {
09     q->num = q->front = q->rear = 0;
10     if((q->que = calloc(max, sizeof(int))) == NULL) {
11         q->max = 0; // 배열 생성에 실패
12         return -1;
13     }
14     q->max = max;
15     return 0;
16 }
17
```

```
/*--- 큐를 구현하는 구조체 ---*/
typedef struct {
    int max; // 큐의 최대 용량
    int num; // 현재의 요소 개수
    int front; // 프런트
    int rear; // 리어
    int *que; // 큐의 맨 앞 요소에 대한 포인터
} IntQueue;
```

(실습 4-5[B]에서 계속)



04-2 큐란?

◆ 인큐 함수 Enque

- 큐에 데이터를 인큐하는 함수
- **a** 처음부터 차례대로 데이터 7개 {3, 5, 2, 6, 9, 7, 1}을 넣은 큐에 8을 인큐
 - ✓ **1** `que[rear](que[2])`에 인큐할 데이터를 저장하고, `rear`와 `num` 값을 1만큼 증가
 - ✓ 만약 인큐하기 전의 `rear`값이 배열의 물리적인 끝(11)이라면 Enque 함수를 수행한 다음에는 `rear` 값이 12가 되면서 `max(12)`와 같아지는 인덱스 초과 문제가 발생
- **b** `rear`값이 11인 큐에 9를 인큐
 - ✓ **2** `rear`값을 1만큼 증가했을 때 큐의 최대 용량의 값인 `max`와 같아지면 `rear`값을 배열의 처음 인덱스인 0으로 변경해야 함

다음 페이지 그림 참고!



04-2 큐란?

◆ 인큐 함수 Enqueue

a 8을 인큐

Enqueue(&q, 8)

num = 7

0	7
1	1
2	
3	
4	
5	
6	
7	3
8	5
9	2
10	6
11	9

num = 8

0	7
1	1
2	8
3	
4	
5	
6	
7	3
8	5
9	2
10	6
11	9

인큐

b 9를 인큐

Enqueue(&q, 9)

num = 7

0	
1	
2	
3	
4	3
5	1
6	2
7	3
8	5
9	2
10	6
11	

num = 8

0	
1	
2	
3	
4	3
5	1
6	2
7	3
8	5
9	2
10	6
11	9

인큐

[그림 4-11] 큐에 인큐하는 과정



04-2 큐란?

◆ 인큐 함수 Enqueue

Do it! 실습 4-5[B]

• 완성 파일 chap04/IntQueue.c

```
01  /*---큐에 데이터를 인큐 ---*/
02  int Enqueue(IntQueue *q, int x)
03  {
04      if (q->num >= q->max)
05          return -1;          // 큐가 가득 참
06      else {
07          q->num++;
08          q->que[q->rear++] = x; 1
09          if (q->rear == q->max) 2
10              q->rear = 0;
11          return 0;
12      }
13  }
14
```

(실습 4-5[C]에서 계속)



04-2 큐란?

◆ 디큐 함수 Deque

- 큐에서 데이터를 디큐하는 함수
- **a** 처음부터 차례대로 데이터 8개 {3, 5, 2, 6, 9, 7, 1, 8}을 넣은 큐에서 3을 디큐
 - ✓ **1** que[front](que[7])에 저장한 값 3을 꺼내고 front값을 1만큼 증가한 다음 num값을 1만큼 감소
 - ✓ 디큐하기 전의 front값이 배열의 물리적인 끝(11)이라면 위의 과정을 거치고 난 후의 front값은 max(12)가 되어 배열 마지막 요소의 인덱스를 초과
- **b** front값이 11인 큐에서 9를 디큐
 - ✓ **2** 1만큼 증가한 front값이 큐의 최대 용량의 값인 max와 같아지면 front값을 배열의 처음 인덱스인 0으로 변경해야 함

다음 페이지 그림 참고!

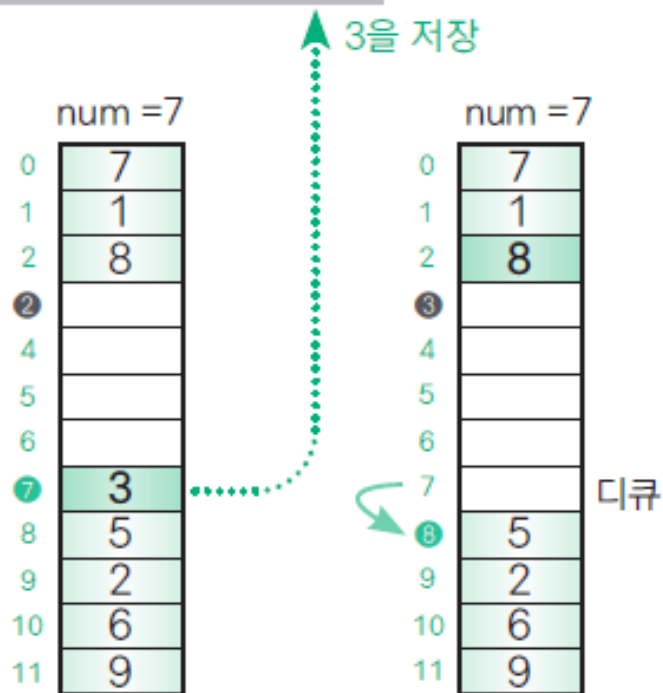


04-2 큐란?

◆ 디큐 함수 Deque

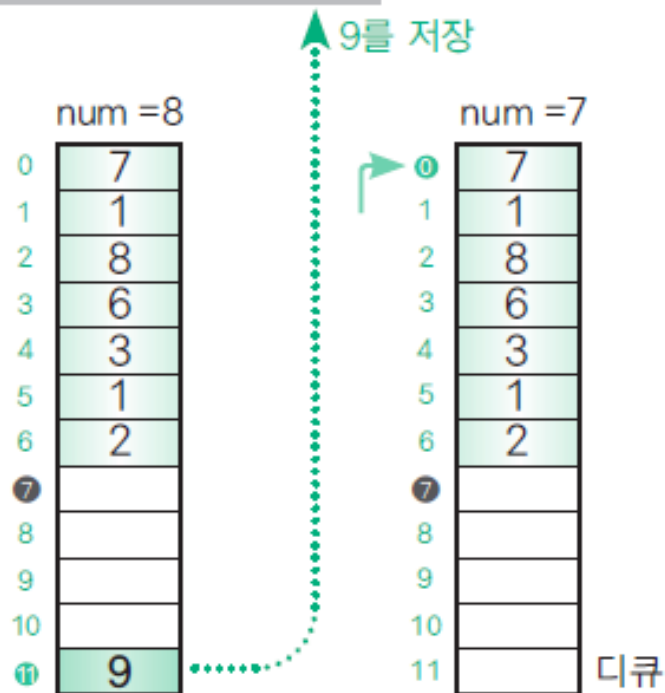
a 디큐

Deque(&q, &x)



b 디큐

Deque(&q, &x)



[그림 4-12] 큐에서 디큐하는 과정



04-2 큐란?

◆ 디큐 함수 Deque

Do it! 실습 4-5[C]

• 완성 파일 chap04/IntQueue.c

```
01  /*--- 큐에서 데이터를 디큐 ---*/
02  int Deque(IntQueue *q, int *x)
03  {
04      if (q->num <= 0)           // 큐는 비어 있음
05          return -1;
06      else {
07          q->num--;
08          *x = q->que[q->front++]; 1
09          if (q->front == q->max) 2
10              q->front = 0;
11          return 0;
12      }
13  }
14
```

(실습 4-5[D]에서 계속)



04-2 큐란?

◆ 피크 함수 Peek

- 앞의 데이터를 '몰래 엿보는' 함수로, `que[front]`의 값을 출력만 하며 `front`, `rear`, `num`의 값이 변하지 않음

◆ 모든 데이터를 삭제하는 함수 Clear

- 현재 큐의 모든 데이터를 삭제하는 함수

◆ 최대 용량을 확인하는 함수 Capacity

- 멤버 `max`값을 그대로 반환

◆ 데이터 개수를 확인하는 함수 Size

- 멤버 `num`값을 그대로 반환



04-2 큐란?

Do it! 실습 4-5[D]

• 완성 파일 chap04/IntQueue.c

```
01  /*--- 큐에서 데이터를 피크 ---*/
02  int Peek(const IntQueue *q, int *x)
03  {
04      if(q->num <= 0)    // 큐는 비어 있음
05          return -1;
06      *x = q->que[q->front];
07      return 0;
08  }
09
10  /*--- 모든 데이터 삭제 ---*/
11  void Clear(IntQueue *q)
12  {
13      q->num = q->front = q->rear = 0;
14  }
15
16  /*--- 큐의 최대 용량 ---*/
17  int Capacity(const IntQueue *q)
18  {
19      return q->max;
20  }
21
22  /*--- 큐에 쌓여 있는 데이터 개수 ---*/
23  int Size(const IntQueue *q)
24  {
25      return q->num;
26  }
```



04-2 큐란?

◆ 큐가 비어 있는지 판단하는 함수 IsEmpty

- 비어 있으면 1, 그렇지 않으면 0을 반환

◆ 큐가 가득 찼는지 판단하는 함수 IsFull

- 가득 찼으면 1, 그렇지 않으면 0을 반환

```
28  /*--- 큐가 비어 있나요? ---*/
29  int IsEmpty(const IntQueue *q)
30  {
31      return q->num <= 0;
32  }
33
34  /*--- 큐가 가득 찼나요? ---*/
35  int IsFull(const IntQueue *q)
36  {
37      return q->num >= q->max;
38  }
```

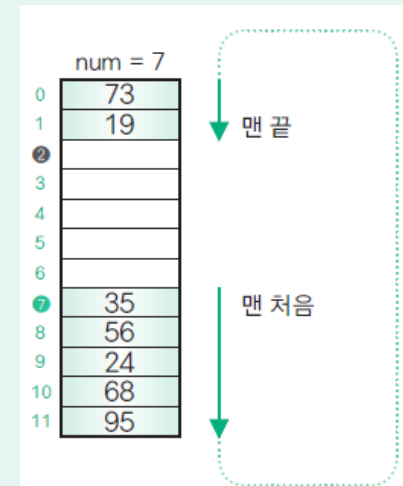


04-2 큐란?

◆ 검색 함수 Search

- 큐의 배열에서 x와 같은 데이터가 저장되어 있는 인덱스를 반환
- 검색에 실패하면 -1을 반환
- 배열의 물리적인 첫요소가 아니라 큐의 논리적인 첫 요소부터 선형 검색을 수행
- 현재 검색하는 위치의 인덱스를 구하는 식은 $(i + q \rightarrow \text{front}) \% q \rightarrow \text{max}$

```
40  /*--- 큐에서 검색 ---*/
41  int Search(const IntQueue *q, int x)
42  {
43      for(int i = 0; i < q->num; i++) {
44          int idx;
45          if(q->que[idx = (i + q->front) % q->max] == x)
46              return idx;           // 검색 성공
47      }
48      return -1;                   // 검색 실패
49  }
```



[그림 4-13] 큐에서 선형 검색



04-2 큐란?

◆ 모든 데이터를 출력하는 함수 Print

- 큐의 모든 데이터를 처음부터 끝까지 순서대로 출력하는 함수

◆ 종료 함수 Terminate

- 메모리 공간에 할당한 배열(큐)을 해제하는 함수

```
51  /*--- 모든 데이터 출력 ---*/
52  void Print(const IntQueue *q)
53  {
54      for(int i = 0; i < q->num; i++)
55          printf("%d ", q->que[(i + q->front) % q->max]);
56      putchar('\n');
57  }
58
59  /*--- 큐의 종료 ---*/
60  void Terminate(IntQueue *q)
61  {
62      if(q->que != NULL)
63          free(q->que);           // 메모리 공간에 할당한 배열 해제
64      q->max = q->num = q->front = q->rear = 0;
65  }
```



04-2 큐란?

◆ 큐만들기 예제

▪ 예제 4-6

Chapt04_예제 4-6/IntQueueTest.c

// int형 큐 IntQueue를 사용하는 프로그램

```
#include <stdio.h>
```

```
#include "IntQueue.h"
```

```
int main(void)
```

```
{
```

```
    IntQueue que;
```

```
    if (Initialize(&que, 64) == -1) {
```

```
        puts("큐의 생성에 실패하였습니다.");
```

```
        return 1;
```

```
    }
```



04-2 큐란?

```
while (1) {
    int m, x;

    printf("현재 데이터 수: %d / %d\n", Size(&que), Capacity(&que));
    printf("(1)인큐 (2)디큐 (3)피크 (4)출력 (0)종료: ");
    scanf("%d", &m);

    if (m == 0) break;
    switch (m) {
        case 1: /*--- 인큐 ---*/
            printf("데이터: "); scanf("%d", &x);
            if (Enque(&que, x) == -1)
                puts("\a오류: 인큐에 실패하였습니다.");
            break;

        case 2: /*--- 디큐 ---*/
            if (Deque(&que, &x) == -1)
                puts("\a오류: 디큐에 실패하였습니다.");
            else
                printf("디큐한 데이터는 %d입니다.\n", x);
            break;
```




04-2 큐란?

case 3: /*--- 피크 ---*/

```
if (Peek(&que, &x) == -1)
    puts("\a오류: 피크에 실패하였습니다.");
else
    printf("피크한 데이터는 %d입니다.\n", x);
break;
```

case 4: /*--- 출력 ---*/

```
    Print(&que);
    break;
}
}
Terminate(&que);
return 0;
}
```



04-2 큐란?

◆ 큐만들기 예제 - 실행 결과

실행 결과

현재 데이터 수: 0/64

(1) 인큐 (2) 디큐 (3) 피크 (4) 출력 (0) 종료: 1

데이터: 1

1을 인큐

현재 데이터 수: 1/64

(1) 인큐 (2) 디큐 (3) 피크 (4) 출력 (0) 종료: 1

데이터: 2

2를 인큐

현재 데이터 수: 2/64

(1) 인큐 (2) 디큐 (3) 피크 (4) 출력 (0) 종료: 4

1 2

큐의 내용을 출력

현재 데이터 수: 2/64

(1) 인큐 (2) 디큐 (3) 피크 (4) 출력 (0) 종료: 2

디큐한 데이터는 1입니다.

1을 디큐

현재 데이터 수: 1/64

(1) 인큐 (2) 디큐 (3) 피크 (4) 출력 (0) 종료: 4

2

큐의 내용을 출력

현재 데이터 수: 1/64

(1) 인큐 (2) 디큐 (3) 피크 (4) 출력 (0) 종료: 3

피크한 데이터는 2입니다.

2를 피크

현재 데이터 수: 1/64

(1) 인큐 (2) 디큐 (3) 피크 (4) 출력 (0) 종료: 4

2

큐의 내용을 출력

현재 데이터 수: 1/64

(1) 인큐 (2) 디큐 (3) 피크 (4) 출력 (0) 종료: 0

「2026년도 1학기 한국성서대학교 AI융합학부」

감사합니다.

